EcoNest: A Housing Market Analytics API

COMP3011 Web Services and Web Data — Technical Report

Student: Yusuf Demir | **Date:** March 2026

-- github repository: https://github.com/lm-yusuf/web-services-final live website: https://web-services-final.vercel.app/login API documentation: https://web-services-final.up.railway.app/api/docs powerpoint slides: https://docs.google.com/presentation/d/1GcqEUZturqzaq-SsH4dAZav6BmBvsJG-/edit?usp=sharing&ouid=117720477378066333630&rtpof=true&sd=true

---

1. Introduction & Project Overview

EcoNest is a full-stack web API and single-page application that transforms raw HM Land Registry Price Paid transaction data into accessible, queryable housing market intelligence. The system ingests real transaction records and exposes them through a RESTful API backed by a PostgreSQL relational database, with an interactive Vue 3 frontend for visualisation.

The motivation behind EcoNest is straightforward: UK house prices are one of the most discussed economic indicators, yet the raw Land Registry CSV files are large, poorly formatted, and inaccessible to anyone without data engineering skills. I wanted to build something that takes this real public data and turns it into something genuinely useful — clean endpoints for price trends, regional heatmaps, affordability metrics, and market signals, all wrapped in an authenticated multi-user experience with saved listings and comparison tools.

Two complete years of Land Registry data (2024 and 2025) were imported — over 500,000 real transaction records. For the current release, API and frontend views are intentionally restricted to **2025-only** outputs so users see a single consistent reporting window.

---

2. Technology Stack Justification

2.1 Backend: Node.js + Express + TypeScript

I chose **Node.js with Express** as the API runtime for practical reasons:

- **TypeScript across the stack**: The most important factor was end-to-end type safety. With TypeScript on both the backend (Express) and frontend (Vue 3), I could share type definitions between layers, catching query shape mismatches and parameter errors at compile time rather than discovering them in production. For a data-heavy analytics API, this matters enormously — a missing field or wrong type in an aggregation result would silently corrupt charts.
- **Familiar stack for verification**: I regularly build with TypeScript and Vue, so choosing this stack meant I could critically review Copilot output instead of accepting it blindly. Familiarity made it easier to spot weak abstractions, incorrect typings, and architectural drift during AI-assisted implementation.
- **Async I/O**: Node's non-blocking event loop handles the IO-bound workload of this API well — it's mostly database reads and HTTP responses, not CPU-intensive computation.
- **Ecosystem depth**: npm has production-ready solutions for everything I needed — Prisma for ORM, Zod for validation, bcrypt for password hashing, swagger-jsdoc for API documentation.
- **Alternatives I considered**: I looked at Python/FastAPI seriously — the automatic OpenAPI generation is excellent. But I'd have lost the shared-type advantage between backend and frontend.

I also briefly considered Go with Fiber for raw performance, but the development speed with TypeScript and the Prisma ecosystem made Node the pragmatic choice.

2.2 ORM: Prisma

Prisma over Sequelize or raw SQL because:

- The **schema-first approach** generates fully typed client code from `schema.prisma`, eliminating query shape bugs at a language level.
- The **migration system** keeps a reproducible history of every schema change — essential when deploying to Railway where I needed migrations to run automatically.
- The **`groupBy` and aggregate APIs** let me express complex analytics queries (average price by county, monthly transaction counts) cleanly without writing raw SQL, avoiding injection risks.

2.3 Database: PostgreSQL

PostgreSQL was the clear choice over SQLite (too limited for 500k+ records) and MySQL:

- **Aggregate functions and window functions** handle the analytics computations efficiently at the database level.
- **Indexing**: I created seven targeted indexes on `property_sales` covering every column used in WHERE clauses — `town_city`, `district`, `county`, `transfer_date`, `property_type`, `price`, and `postcode`. This keeps query times fast even as data grows.
- **ACID and unique constraints**: The `UNIQUE` constraint on `transaction_id` ensures the import script is idempotent — I can re-run it safely without duplicating records.
- Deployed as PostgreSQL 17 via **Railway's managed Postgres**.

2.4 Frontend: Vue 3 + Pinia + Tailwind CSS + Vite

- **Vue 3 Composition API**: I considered React but chose Vue for its more intuitive reactivity model and gentler learning curve. I also use Vue regularly, which made me more confident auditing Copilot-generated components and store logic. The Composition API provides clean separation of concerns in complex views like the dashboard which loads multiple data sources in parallel.
- **Pinia**: Five stores implemented — `auth`, `comparison`, `dataCache`, `filters`, and `saved`. The `dataCache` store was an important design decision: it memoises API responses with a 5-minute TTL so switching between views doesn't trigger redundant backend calls.
- **Tailwind CSS**: Utility-first approach let me build a consistent dark-themed responsive UI without maintaining a separate CSS design system.
- **Vite**: Much faster hot module replacement than webpack, which made the development feedback loop noticeably quicker.

2.5 Validation: Zod

All incoming request bodies pass through a **Zod schema middleware** before reaching controllers. This validates at the system boundary (where untrusted user input enters) without over-engineering validation throughout internal business logic.

---

3. System Architecture

EcoNest follows a layered architecture with clear separation of concerns:

```
Client (Browser)
     │
     ▼
```

```
Vue 3 SPA (Vercel CDN)
     │  HTTPS / Bearer token
     ▼
Express API (Railway)
  ├── Routes (URL → Controller mapping)
  ├── Middleware (Auth, Validation, Error handling)
  ├── Controllers (HTTP adapter layer)
  └── Services (Business logic & DB queries)
          │
          ▼
     Prisma ORM (Type-safe query builder)
          │
          ▼
   PostgreSQL 17 (Railway Managed)
```

Key architectural decisions and why I made them:

- **Controller–Service split**: Controllers are thin HTTP adapters — they extract request parameters and map service errors to HTTP status codes. All actual business logic lives in service classes (`PropertyService` , `MarketInsightsService` , `AuthService` , `SavedListingService`). This means I can test service logic independently of the HTTP layer, and the services are reusable if I ever add a GraphQL or WebSocket layer.
- **Database-backed sessions instead of JWTs**: This was a deliberate choice I debated during development. JWTs are stateless and simpler to implement, but they cannot be revoked — if a user logs out, the JWT remains valid until expiry. With DB-backed sessions, logout immediately deletes the session row. The trade-off is one extra database read per authenticated request, but the indexed `token` column keeps this fast.
- **CORS configuration**: Because the frontend (Vercel) and backend (Railway) are on different domains, CORS had to be configured carefully. I use a whitelist of explicit origins plus a regex pattern (`^https://[a-z0-9-]+\.vercel\.app$`) to accept Vercel's auto-generated preview deployment URLs, so I don't have to manually whitelist every preview.
- **Global error handler**: A single `errorHandler` middleware catches all unhandled exceptions and formats them into a consistent `{ success: false, error: "..." }` envelope. Stack traces are never sent to clients — this prevents information leakage.

---

4. API Design & Data Modelling

4.1 Endpoint Summary

The API has four route groups, all under `/api/` :

Group	Endpoints	Auth Required
Auth	`POST /auth/register`, `POST /auth/login`, `POST /auth/logout`, `GET /auth/me`	Login/me/logout
Properties	`GET /trends`, `GET /heatmap`, `GET /properties`, `GET /regions`, `GET /stats`	No
Saved Listings	`GET /saved`, `POST /saved`, `PUT /saved/:id`, `DELETE /saved/:id`	Yes

Market Insights	`POST /insights/region`	No

Every response uses a consistent envelope: `{ "success": true/false, "data": ... }`. HTTP status codes follow RFC 7231: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 409 Conflict, 500 Internal Server Error.

Interactive documentation is available via **Swagger UI** at `/api/docs`, with a raw OpenAPI JSON spec at `/api/docs.json`. Every endpoint has JSDoc annotations that feed into swagger-jsdoc.

4.2 Analytics Endpoints

The `/trends` endpoint fetches filtered transactions from PostgreSQL, groups them by year-month in the application layer, and derives month-over-month price growth:

To keep all user-facing analytics consistent, the backend enforces a fixed `transferDate` range for 2025. The trends year dropdown is therefore locked to `2025`, and manual requests for other years are ignored by the API.

$$priceGrowth% = \frac{avgPrice_t - avgPrice_{t-1}}{avgPrice_{t-1}} \times 100$$

I chose in-application grouping here rather than a SQL GROUP BY because Prisma's API makes month-key construction cleaner in TypeScript than in raw SQL date formatting.

The `/insights/region` endpoint is the most complex — it fires **nine parallel database aggregations** using `Promise.all()` to build a comprehensive market profile: overall price statistics, recent-year vs previous-year averages (for one-year growth), five-year window comparison, property type breakdown with count and average price per type, tenure split, new-build share, and a market signal classification. The affordability ratio divides average price by £35,000 (approximate UK median household income):

$$affordabilityRatio = \frac{avgPrice}{35000}$$

The market signal classifies regions as `strong-growth` (≥5% YoY), `moderate-growth` (≥1%), `stable` (≥-1%), `declining` (<-1%), or `insufficient-data` (<10 transactions).

4.3 Data Model

Six Prisma models map to PostgreSQL tables:

- **`property_sales`**: Core analytical table — maps Land Registry CSV fields (transaction ID, price, date, postcode, property type, tenure, location) to typed columns with 7 indexes.
- **`users`**: Email + bcrypt password hash. UUIDs as primary keys (no sequential ID leakage).
- **`sessions`**: Token-based auth with indexed token column and expiry timestamp. CASCADE delete on user removal.
- **`saved_listings`**: Many-to-many link between users and properties with a `UNIQUE(user_id, property_id)` constraint preventing duplicate saves.
- **`market_reports`**: Defined for caching pre-computed analytics (not yet populated — noted as future work).
- **`user_preferences`**: Stores favourite regions, budget range, and preferred property types per user.

All foreign keys use CASCADE deletes — deleting a user automatically removes their sessions, saved listings, and preferences.

---

5. Security Implementation

Security was not an afterthought — it was built into every layer:

- **Password hashing**: bcrypt with 12 salt rounds. Plaintext passwords are never stored, logged, or returned in any response.
- **Token generation**: `crypto.randomBytes(48)` produces 384 bits of cryptographic entropy per session token — computationally infeasible to brute-force.
- **Session expiry**: 7-day TTL enforced server-side. The auth middleware rejects expired tokens even if the token string is valid, and cleans up the expired row.
- **Input validation**: Zod schemas on all mutating endpoints prevent malformed data from reaching the database. Prisma's parameterised queries provide a second defence layer against SQL injection.
- **User enumeration prevention**: Login failure returns the same generic message ("Invalid email or password") regardless of whether the email exists — an attacker cannot distinguish between a wrong email and a wrong password.
- **CORS**: Strict origin whitelist plus the Vercel regex pattern. No wildcard `*` origins.
- **Error handling**: Global error handler prevents stack trace leakage. Internal errors return a generic message to clients.

---

6. Testing Strategy

Tests use **Vitest** and **Supertest** for full HTTP-layer integration testing against a real database:

- **Auth tests** (`auth.test.ts`): Registration, duplicate email rejection (409), invalid input validation (400), login, token-based `/me` access, logout invalidation, and unauthenticated access rejection on protected routes.
- **Property tests** (`properties.test.ts`): Trend queries with and without filters, heatmap data shape validation, paginated property listing with price/type/region filters, region list, and stats endpoint.
- **Saved listings tests** (`savedListings.test.ts`): Auth guard validation, invalid UUID validation, duplicate-save conflict handling (409), empty-list reads, update/delete not-found paths, and owner-only access behavior for protected routes.
- **General tests** (`general.test.ts`): Health check, 404 handling for unknown routes.

I deliberately chose integration tests over unit tests. In a data API, the most common bugs occur at the boundary between HTTP input parsing, service logic, and database queries. Mocking the database would hide the exact class of bugs most likely to occur — malformed Prisma queries, missing field mappings, incorrect aggregation logic. Integration tests running the full pipeline catch these.

---

7. Deployment & Operations

Component	Platform	URL
Backend API	Railway	Auto-deploy from GitHub `main`
PostgreSQL 17	Railway Managed	Internal URL at runtime; external URL for imports
Frontend SPA	Vercel	Auto-deploy from GitHub with preview deployments

Deploy pipeline: Pushing to `main` triggers both Railway and Vercel builds. Railway runs `npm install && npm run build` , then `npx prisma migrate deploy && npm start` (configured in `railway.json`).

Vercel builds the Vue app and serves it from CDN with SPA fallback routing (`vercel.json`).

Environment variables: `DATABASE_URL` , `CORS_ORIGIN` , and `PORT` on Railway; `VITE_API_BASE_URL` on Vercel pointing to the Railway backend URL.

Data import: A dedicated CLI script (`scripts/importPriceData.ts`) reads Land Registry CSVs, streams and parses records in batches of 500, and uses `createMany` with `skipDuplicates: true` for idempotent import. The script handles both named-column and positional CSVs, and falls back to individual inserts if a batch fails. I ran imports locally against Railway's external database URL for the 2024 and 2025 datasets, with presentation-layer/API filtering currently pinned to 2025.

Health check: `/api/health` returns `{ status: "ok", timestamp: "..." }` — Railway uses this as a liveness probe.

---

8. Challenges & Lessons Learned

Deployment struggles

Deployment was the single most frustrating part of this project. Getting the code to work locally was relatively straightforward, but making it work across Railway and Vercel involved multiple issues I had to debug one by one:

- **Railway build failures**: My first several deploys failed because Railway's Nixpacks builder couldn't resolve the TypeScript build correctly. I had to experiment with the `railway.json` configuration — getting the build command (`npm install && npm run build`) and start command (`npx prisma migrate deploy && npm start`) right took multiple deploy-and-check cycles. There's no local way to exactly replicate Railway's build environment, so I was reading deploy logs remotely each time.
- **Database connectivity**: Railway provides both an internal URL (for services within Railway's network) and an external/public URL (for connecting from outside). I initially tried to use the internal URL for data imports from my local machine, which obviously didn't work. Understanding the difference and knowing when to use which URL took some trial and error.
- **CORS preflight failures**: After deploying both the frontend (Vercel) and backend (Railway), the first thing that broke was CORS. The browser was sending preflight OPTIONS requests that the backend rejected because the Vercel deployment URL wasn't in the allowed origins list. Worse, Vercel generates a unique subdomain for each deployment (`https://<hash>.vercel.app`), so I couldn't just whitelist one URL. I ended up implementing a regex check in the CORS origin callback to accept any `*.vercel.app` subdomain — but getting to that solution involved hours of reading browser console errors and Railway logs.
- **Data import to production**: Importing 500k+ records to Railway's managed Postgres via the external URL was slow and occasionally timed out. I had to ensure the import script was resilient to connection drops and that batch sizes were tuned to avoid overwhelming the remote database.

Data volume and performance

The Land Registry dataset is large — hundreds of thousands of records per year. Early versions of the analytics endpoints were noticeably slow before I added proper indexes. I added seven single-column indexes on `property_sales` covering every column used in WHERE clauses. The decision of whether to aggregate in the database (heatmap groupBy) or in the application (trend month-over-month calculations) was driven by what Prisma's API could express cleanly — in-DB for simple groupBy, in-app for derived calculations like percentage growth.

Session architecture

I initially went with JWTs because they're simpler — no database table needed. But during development I realised that a real logout feature requires token revocation, which JWTs inherently cannot provide without maintaining a blocklist (defeating the stateless advantage). Switching to database-backed sessions added complexity (the `sessions` table, the auth middleware doing a DB lookup per request, the indexed token column) but gave me genuine instant logout and the ability to see/manage active sessions.

TypeScript strictness catching real bugs

Running TypeScript in strict mode caught several bugs at compile time that would have been subtle runtime issues — particularly around optional query parameters being used as strings without null checks, and Prisma query results having potentially null fields in aggregations.

---

9. Limitations & Future Development

- **Median price approximation**: The current insights endpoint approximates the median rather than using SQL `PERCENTILE_CONT`, because Prisma doesn't natively support percentile aggregates. A raw query would give a true median.
- **No rate limiting**: Public endpoints have no rate limiting. Adding `express-rate-limit` is essential before any real public exposure.
- **Static dataset**: Data is a one-time CSV import. A scheduled cron job could pull monthly updates from the Land Registry's published data.
- **County-level heatmap only**: The heatmap aggregates by county name (text). Integrating PostGIS with coordinates would enable real map-based visualisation with polygon overlays.
- **MarketReport cache**: The schema defines a `market_reports` table for caching pre-computed analytics, but it isn't populated yet. Redis or materialised views could also serve this caching role.
- **No WebSocket support**: All data is request-response. Real-time price alerts or live dashboard updates would require WebSocket integration.

---

10. GenAI Declaration Summary

Full declaration: See `GENAI_DECLARATION.md` for the complete, detailed breakdown.

Tools used: Google Gemini, ChatGPT (OpenAI), GitHub Copilot (Claude Sonnet 4.6) via VS Code.

I used three AI tools in a sequential pipeline, each for a specific purpose:

```
[Me] → Gemini (ideation + requirements) → ChatGPT (prompt engineering) →
Copilot/Claude (implementation)
```

How it worked in practice:

1. I opened a conversation with **Gemini** to explore project ideas from the brief. After about 15 minutes of discussion, I settled on the housing market analytics API using Land Registry data. I then continued the Gemini conversation to produce a structured requirements analysis — functional requirements, non-functional requirements, data model, frontend flows — given the technology stack I had decided on.

2. I took Gemini's requirements document and gave it to **ChatGPT** with a specific task: convert these requirements into a detailed, precise prompt that I could give to a code-generation AI. ChatGPT's

role was pure prompt engineering — structuring the instruction so that file layout, type definitions, layering conventions, security requirements, and test expectations were all explicit.

3. I gave the ChatGPT-engineered prompt to **GitHub Copilot (Claude Sonnet 4.6)** in VS Code. Copilot built the project iteratively — schema first, then services, controllers, routes, tests, and finally the frontend. Throughout this process I was actively involved: reviewing every piece of generated code, challenging architectural decisions (pushing back on JWT in favour of sessions), feeding test failures back for debugging, and making manual adjustments where the generated code didn't match what I wanted.

4. Copilot also helped with **debugging deployment issues** (the CORS preflight failure with Vercel preview URLs), **reviewing test coverage** (identifying missing edge cases), and **structuring this report and presentation**.

Reflective analysis:

The most distinctive aspect of my GenAI usage was the multi-model pipeline. Starting with Gemini for ideation meant I spent approximately 15 minutes in structured dialogue before committing to a project — exploring trade-offs between different domain areas (transport, nutrition, housing) and receiving domain-specific reasoning about dataset quality and analytical depth. This is a qualitatively different use of AI than using it reactively during coding.

The ChatGPT prompt-engineering step was equally deliberate. Rather than feeding Gemini's output directly to a code-generation tool, I used ChatGPT's strength in instruction optimisation to convert a broad requirements document into a build prompt with precise specifications for file structure, type interfaces, layering rules, security requirements, and test coverage expectations. The quality of the resulting Copilot build was directly attributable to the quality of this prompt.

During implementation, Copilot was used as an active technical interlocutor, not just a code writer. Its challenge to the JWT approach led to the DB-backed session design. Its observation that housing prices are right-skewed (making median more meaningful than mean) led to the `medianPrice` field in the insights endpoint. All generated code was reviewed, tested, and adapted. Prisma query syntax was cross-checked against official documentation where the AI showed overconfidence.

This pipeline represents the "high-level" GenAI usage band: creative application of multiple AI tools to explore alternatives, engineer solutions, and reimagine the development workflow itself.

References

- HM Land Registry (2024–2025). *Price Paid Data*. Available at: https://www.gov.uk/government/collections/price-paid-data
- Prisma. (2024). *Prisma Documentation*. Available at: https://www.prisma.io/docs
- OWASP. (2023). *OWASP Top Ten*. Available at: https://owasp.org/www-project-top-ten/
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- Vue.js. (2024). *Vue 3 Documentation*. Available at: https://vuejs.org/guide
- Railway. (2024). *Railway Documentation*. Available at: https://docs.railway.app